

Verified processes on the BEAM

What falls out when a receive loop is a transition relation, code is a database, and the solver is already in the box

prototyped — all eight spikes green, pre-MVP

2026-08-29

The one idea

A PROCESS IS A TRANSITION RELATION YOU ALREADY WROTE

A BEAM process loop is a state machine hiding in plain sight. Read this:

```
(loop [n n]
  (receive
    [:inc]          (recur (+ n 1))
    [:dec] :when (> n 0) (recur (- n 1))
    [:reset]        (recur 0)))
```

 Clojure

The loop variable `n` is the **state**. Each `receive` clause is a **transition**: the pattern and `:when` are its enabling condition, the body's `recur` names the next state. The set of clauses is a disjunction — TLA⁺'s $\text{Next} \triangleq \bigvee A_i$, written not in a separate file in a separate language, but as the running code itself.

TLA⁺ makes you write that **Next** relation **beside** your code and then **hope** the implementation matches it. That hope is the refinement gap — the crack every real specification falls through. Here there is no gap to fall through, because the specification **is** the code, extracted as data.

This document is the “what becomes possible” tour. Everything in it is prototyped and runs today; each claim carries the demo that proves it. The thesis in one line:

Verification is a property of the beam-lisp **evaluator**, not of any macro or framework. Write an ordinary server; it becomes legible to a checker that reads the primitive layer it already compiles to.

Because — the guarantees attach to `receive` / `spawn` / `send` / `loop - recur` / function clauses / guards / `defmulti` — the primitives every process is built from — so nothing needs rewriting to earn them

Why native, and why not a JVM tool

The obvious move is to reach for TLA⁺'s tooling. We do not, for one decisive reason: those tools are **encoders**. Apache's whole pipeline — parse TLA⁺ text, type-check, normalize, encode to

SMT — exists to get from *TLA⁺ source text* to a solver. We have no text: our `Next` relation is already beam-lisp data. And we already own the solver (z3, bundled in `priv/z3`, driven as a native call at 2 ms per query since MVP-C).

Reimplement the encoder natively; keep z3. This is the exact line the project already drew — baby-SMT was rejected (don't reimplement the **solver**), z3 was blessed, and every other encoder (the type checker, datalog, miniKanren) was built native. A model checker's front half is an encoder. It falls on the reimplement side.

Because — Apache is 196 MB of JVM whose reason to exist is a translation we do not need; z3 is the one part worth keeping, and we already have it

We can still **emit** TLA⁺ as an export for anyone wanting a second opinion from TLC — but as a pretty-printer, never a dependency.

The substrate: three engines, already here

Nothing below is new machinery. The system-model tier is the type system's own tools pointed one level up — from “functions and calls” to “states and transitions.”

owns	does
tag lattice	structure — state shapes, effect footprints
datalog (native)	reachability, cycles, the whole-program call graph
z3 (native)	arithmetic decisions — invariants, simulation, guard overlap
miniKanren (native)	synthesis — inhabitants, and abducted preconditions

The division is strict and never overlaps — the project's standing rule. The payoff appears immediately: extracting a transition relation is not new parsing. It reuses `parse-clause`, `defn-clauses`, and the guard-to-tag table the type checker already built. The clause / guard / dispatch machinery written for **type-checking** *is* the model extractor.

Extraction: five spellings, one machine

The counter above, written as a raw `receive` loop, as an OTP `defserver`, and as a `defmulti` dispatch, all lower to the identical transition graph.

```
$ BeamLisp.run_file("research/p16_model/run.bl")
```

```
raw receive-loop ([[:dec "(- n 1)"] [:inc "(+ n 1)"] [:reset "0"]])
defserver (OTP)  ([[:dec "(- n 1)"] [:inc "(+ n 1)"] [:reset "0"]])
defmulti dispatch ([[:dec "(- n 1)"] [:inc "(+ n 1)"] [:reset "0"]])
all three identical: true
```

`defserver` $\equiv$ `loop` holds *exactly*. An OTP server, written with no verification in mind, is the same machine as a raw `receive` loop — so it inherits the whole guarantee battery with zero edits. This is the crux: the guarantees are not bolted onto a special form; they belong to the primitive every form lowers to.

A detail that matters for honesty: the extractor reports each transition’s guard *where the source encodes it* — `:when` for the loop and server, a body `(if ...)` for the `defmulti` — and never fabricates one. Sound-warnings discipline, now at the model layer.

Safety, proven unbounded

A finite search can only check the states it enumerates. `z3` lets us make the leap to proof. For an invariant `Inv` over the state, discharge the Hoare triple

$$\text{Inv}(s) \wedge \text{guard}(s) \wedge s' = \text{next}(s) \vdash \text{Inv}(s')$$

for *every* state and *every* input at once — assert its negation, check for unsatisfiability. If `init` establishes the invariant and every transition preserves it, then $\Box \text{Inv}$ holds by induction over an unbounded state space.

```
$ BeamLisp.run_file("research/p16_model/run_inductive.bl")
```

```
— claim:  $\Box(\text{balance} \geq 0)$  holds – proven, unbounded —
init establishes  $\text{balance} \geq 0$ : true
deposit preserves  $\text{balance} \geq 0$ : true
withdraw preserves  $\text{balance} \geq 0$ : true
 $\Box(\text{balance} \geq 0)$  proven by induction. No bound on state or input.

— bug B1: unguarded withdraw → z3 returns the overdrawing state —
counterexample (state+amount that overdraws):
amt=1, balance=0, balance2=-1
```

The checker does not merely say “unsound.” It hands back the concrete state and input that break the invariant — `balance=0, amt=1 → -1`, the exact off-by-one class TLA^+ is famous for catching, caught here from the source with no separate spec. This is a direct extension of MVP-C: rewrite-equivalence there, state-invariant preservation here, the same `z3` port. `beam-lisp`’s prefix source is already valid SMT-LIB arithmetic, so the extracted transition becomes a proof obligation almost verbatim.

When the invariant is not inductive: abduction

Users write invariants that are true yet not *inductive* — the one-step triple fails because it quantifies over states the guard never reaches. TLA^+ users know this ritual as “strengthen the inductive invariant,” and they do it by hand. We automate it, and it is the cleanest illustration of the three engines cooperating with zero overlap.

ABDUCTION = ENUMERATE, DECIDE, DISCHARGE

To find the missing precondition P closing the triple:

- **miniKanren** enumerates candidate predicate structures over the state and input variables (this is the backwards synthesis of FEAT-027, pointed at predicates instead of programs).
- **z3** decides which candidates actually make the triple valid.

- **datalog** discharges the survivor against the call graph: do *all* senders already guarantee *P*? If so, it needs no annotation at all.

```
$ BeamLisp.run_file("research/p16_model/run_abduce.bl")
```

```
— rung 2: unguarded withdraw – abduce the missing precondition —
candidates enumerated by miniKanren, decided by z3.
shortest sufficient P: (<= amt 0) (a non-positive withdrawal cannot overdraw)

— rung 3: datalog discharge – do all senders already guarantee P? —
every caller of :withdraw guarantees (>= balance amt): true
→ NO annotation needed.
after adding an unguarded caller: still discharged? false
→ the one careless call site is found.
```

Rung 3 is the move *only we can make*. TLA⁺ has no callers to query — it has no implementation. We have both the specification and every call site in the same database, so “every sender already establishes *P*” is a join, and the precondition can discharge with no annotation whatsoever. And the search is honest about its limits: give it too small a vocabulary and it returns nothing rather than a false proof — the domain is a knob, and its insufficiency is reported, never hidden.

Effects, redefined: a footprint

The shipped effect model is a rung on a ladder — `pure < atom < process < io`. That is a lossy summary. To reach separation-logic-grade reasoning, an effect becomes what it actually is: a set of resources, each touched in a mode.

$$\text{effect} \triangleq \{\text{resource} \mapsto \text{mode}\}, \quad \text{mode} \in \{R, A, W, S, K, X\}$$

Read (R), append/monotone (A), write (W), send (S), receive (K), spawn (X). The ladder is recovered as `rank(join of modes)` — nothing regresses — and the algebra the footprint carries *is* the guarantees.

```
$ BeamLisp.run_file("research/p16_model/run_footprint.bl")
```

```
(datum/transact! conn tx) footprint {"conn" :A} monotone? true
(swap! cache assoc k v)    footprint {"cache" :W} monotone? false
swap!a × swap!b (disjoint): commute? true
swap!a × swap!a (shared W): commute? false
:orders handler × :balance handler: frame-independent? true
```

Three properties fall straight out of the footprint:

- **Monotonicity** is a footprint touching only R and A. `datum/transact!` is append-only, which is *exactly* why a live query over it is safe — MVP-F’s monotone-live-query gate is now this single predicate.
- **Commutativity** is the absence of a shared resource with a conflicting mode. Disjoint writes commute; two writes to the same atom do not. Confluence proven by algebra, not by exploring interleavings — the model checker’s most expensive job, avoided.

- **The frame rule** — Separation Logic’s core — is disjointness of resources. The `:orders` handler *cannot* break an invariant over `:balance`, because their footprints do not intersect. Local reasoning, no annotation, no whole-state proof. This is what lets verification scale past toy servers.

Resources we cannot name — a runtime pid, message contents — collapse to a single `:opaque-world` top that conflicts with everything: precise where we can name, sound where we cannot.

The sandbox, unified

beam-lisp already has a capability sandbox: an environment is a world of callable code, a fork can only narrow (`child = parent n spec`), and code cannot even *name* what it was not granted. The insight of this tier is that capabilities and effect-footprints are the *same lattice*, seen from opposite ends.

GRANTED CAPS — THE CEILING
what code *may* touch. An *upper* bound, enforced at compile time, narrowed by every fork.

INFERRED FOOTPRINT — THE FLOOR
what code *does* touch. A *lower* bound, computed by effect inference.

Verifying a sandboxed process is then one containment: `footprint  $\subseteq$  caps`. The sandbox already enforces the upper bound; effect inference supplies the lower. And the consequences are exactly the security properties the sandbox documentation states as operator rules — now machine-checked.

```
$ BeamLisp.run_file("research/p16_model/run_sandbox.bl")
```

```
payoff 1 disjoint caps  $\Rightarrow$  disjoint footprints  $\Rightarrow$  all A-workers commute
          with all B-workers — a fleet's race-freedom from the fork tree,
          zero per-pair checks.
payoff 2 (erlang/spawn worker) with no convey  $\rightarrow$  escapes the cap set  $\rightarrow$  flagged.
payoff 3 destroyed env  $\rightarrow$   $\emptyset$  caps  $\rightarrow$  a worker touching anything is caught;
          one touching nothing is provably inert.
payoff 4 an unverified :global call is :opaque-world  $\rightarrow$  the confused-deputy
          hole becomes a discharged proof obligation.
```

The fork tree *is* a fleet non-interference proof. Two tenants forked with disjoint caps have disjoint footprints *by construction*, so every worker of one commutes with every worker of the other — a thousand-worker fleet’s race-freedom, for free, from the shape of the fork tree. The effect model does not duplicate the capability system; it strengthens it.

Deadlock, structurally

A synchronous `call` blocks the caller until the callee replies. A cycle in the resulting “waits-for” graph is a deadlock. That graph is datoms — one `:waits/on` edge per synchronous call site; async casts contribute none, because they never block — and a cycle is `A reachable from A`, a recursive datalog query on the shipped fixpoint engine. No z3, no state exploration.

```
$ BeamLisp.run_file("research/p16_model/run_deadlock.bl")
```

```
orders-billing-shipping-orders (a cycle), + orders-audit
  deadlocked servers: ("billing" "orders" "shipping")
→ the three cycle members named; audit (acyclic) excluded.
make the back-edge async → deadlocked servers: [] (fix verified)
```

TLA⁺ reports a deadlock *state* after searching the space; we report the structural *cycle* — the actual cause, the servers named — straight from the call graph. And breaking the cycle (turning one call async) is confirmed by the same query.

Liveness, and the honesty of a bound

Safety we prove unbounded. Liveness — “every request eventually replies” — is a statement about infinite behaviours, and the practical method is bounded model checking: search for a lasso (a reachable loop that never makes progress) up to depth k . A found lasso is a real violation, with a witness. No lasso within k is a guarantee *up to* k — and that is not a proof.

A bounded result is registered as an **approximate** relation. “Which of my guarantees are exact, and which are bounded?” is itself a query.

Because — a guarantee that misrepresents its own strength is worse than no guarantee; the bound belongs in the catalog, queryable, not in a footnote

```
$ BeamLisp.run_file("research/p16_model/run_liveness.bl")
```

```
buggy handler (:retry spins): lasso found at depth 2, trace [:idle :pending :pending]
approximate guarantees: (:~replies-eventually) ; bound 20
:~balance-nonneg extension: exact ; proven unbounded (inductive)
```

The bounded liveness result sits in the catalog as approximate; the inductive safety result as exact. Honesty is structural.

The crown: verified hot code upgrade

Refinement is the deepest idea in the specification world: concrete C *implements* abstract A when every observable behaviour of C is a behaviour of A , proven by a simulation relation $R(a, c)$ that related states preserve. When C refines A , every property proven on A transfers to C for free. It buys three things, in rising order of “this is why it matters.”

```
$ BeamLisp.run_file("research/p16_model/run_refine.bl")
```

```
payoff 1 a batched counter refines the simple one → balance ≥ 0 transfers free.
payoff 2 a connection refines the open/close protocol → it can never emit
          a message out of order. The abstract machine IS the session type.
payoff 3 VERIFIED HOT UPGRADE (below).
```

The third is one almost no system checks. The BEAM's signature feature is hot code upgrade: swap v2 into a running process *without stopping it*. The danger is `code_change` — the callback that migrates the live state from v1's shape to v2's. Get it wrong and a process that has been correct for months corrupts itself mid-flight, with no restart to catch it.

CODE_CHANGE IS JUST ANOTHER TRANSITION

A hot upgrade is a transition like any other, and the simulation relation must respect it. Take a server upgraded from `{:balance}` to `{:balance :currency}`:

```
;; v1
(defserver account ^{:invariant (fn [s] (>= (:balance s) 0))}
  (handle-call [:withdraw amt] [_ s]
    (if (>= (:balance s) amt) (reply :ok (update s :balance - amt))
      (reply :insufficient s))))

;; v2 — same invariant, a new field, and a migration
(defserver account ^{:refines account-v1}
  (code-change [old] (ok {:balance (:balance old) :currency :usd}))
  (handle-call [:withdraw amt] [_ s] ...))
```

Three obligations, all discharged by z3:

1. the migration establishes v2's invariant from v1's:
`balance ≥ 0 ⊢ migrate(state) has balance ≥ 0`;
2. v2's `withdraw` *simulates* v1's — the observable balance moves identically;
3. therefore every property proven on v1 survives the swap.

```
$ BeamLisp.run_file("research/p16_model/run_refine.bl")
```

```
— payoff 3: VERIFIED HOT UPGRADE — code_change preserves the invariant —
migration b2 := b1 preserves balance ≥ 0: true
v2.withdraw simulates v1.withdraw (observable identical): true
a migration that zeroes the balance is caught: true
```

The hot upgrade is verified end to end. `code_change` preserves the simulation relation, v2 simulates v1, and `balance ≥ 0` — proven on v1 — survives the live swap. A *wrong* migration, one that silently zeroes the balance, is caught before it ships. Correctness *across* a hot code change, with the process never stopping. This is the guarantee the BEAM's most distinctive feature has always lacked.

The honest scorecard

guarantee	strength	how, and against TLA ⁺
state-type	exact	the state shape, from P8/P9 — free on any server
message coverage	exact	datalog join of senders × handled patterns — <i>TLA⁺ cannot: it has no callers</i>
dispatch determinism	exact	guard disjointness, z3 at the leaf
effect footprint	exact	purity / monotone / commute / frame, structural

inductive safety	exact, unbounded	z3 Hoare triple — equals TLAPS, automated
auto-strengthening	best-effort	z3 + miniKanren + datalog, honest when it gives up
deadlock-freedom	exact	a datalog cycle query — the cause, not a state
liveness	bounded	BMC lasso — registered <i>approximate</i> , never overclaimed
refinement	exact	z3 simulation — optimization, protocol, <i>hot upgrade</i>

“More than TLA⁺” resolves to something precise: the same safety, with zero refinement gap because the model is extracted from the code; plus whole-program message-coverage that a specification without an implementation structurally cannot check; minus native temporal liveness, which we bound honestly and can hand to TLC gap-free by export. Every row of this table is prototyped and runs today, on the bundled z3, with no JVM anywhere.

All demos: `research/pl6_model/`. Run with the direct harness
`(elixir -pa _build/dev/lib/*/ebin -e 'BeamLisp.init() ...')` — the eight drivers
`run.bl`, `run_inductive.bl`, `run_abduce.bl`, `run_footprint.bl`, `run_sandbox.bl`,
`run_deadlock.bl`, `run_liveness.bl`, `run_refine.bl` each print their own PASS lines. Plan:
`!tasks/plans/PLAN-048`. Decision record: `DEC-pl6-system-model-tier`.